MiniMax M3: Frontier Coding, 1M Context, Native Multimodality — All in One Model

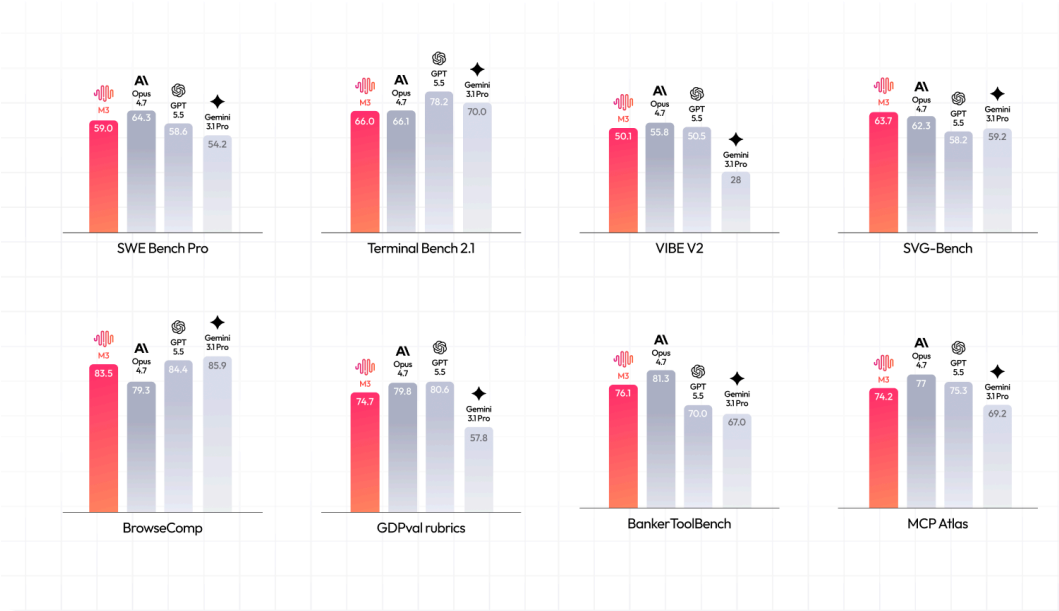
2026-06-01

AI M3 Frontier Model MSA

MiniMax M3 is officially released today.

M3 reaches frontier-level performance on specialized tasks such as coding and agentic work. It uses MSA (MiniMax Sparse Attention), a new attention architecture proposed by our team, and supports ultra-long context windows of up to 1M tokens. To much anticipation, it is also a natively multimodal model that supports image and video input and can operate a desktop computer.

These three capabilities are now table stakes for closed-source frontier models. M3 is currently the first and only open-weight model to bring all three together.



In terms of Coding capabilities, M3 shows significant improvements over M2, approaching the level of leading overseas closed-source models in areas such as bugfix, frontend/backend development, and performance optimization.

domain.

You can experience MiniMax M3 right away through MiniMax Code, the Token Plan, and our API services.

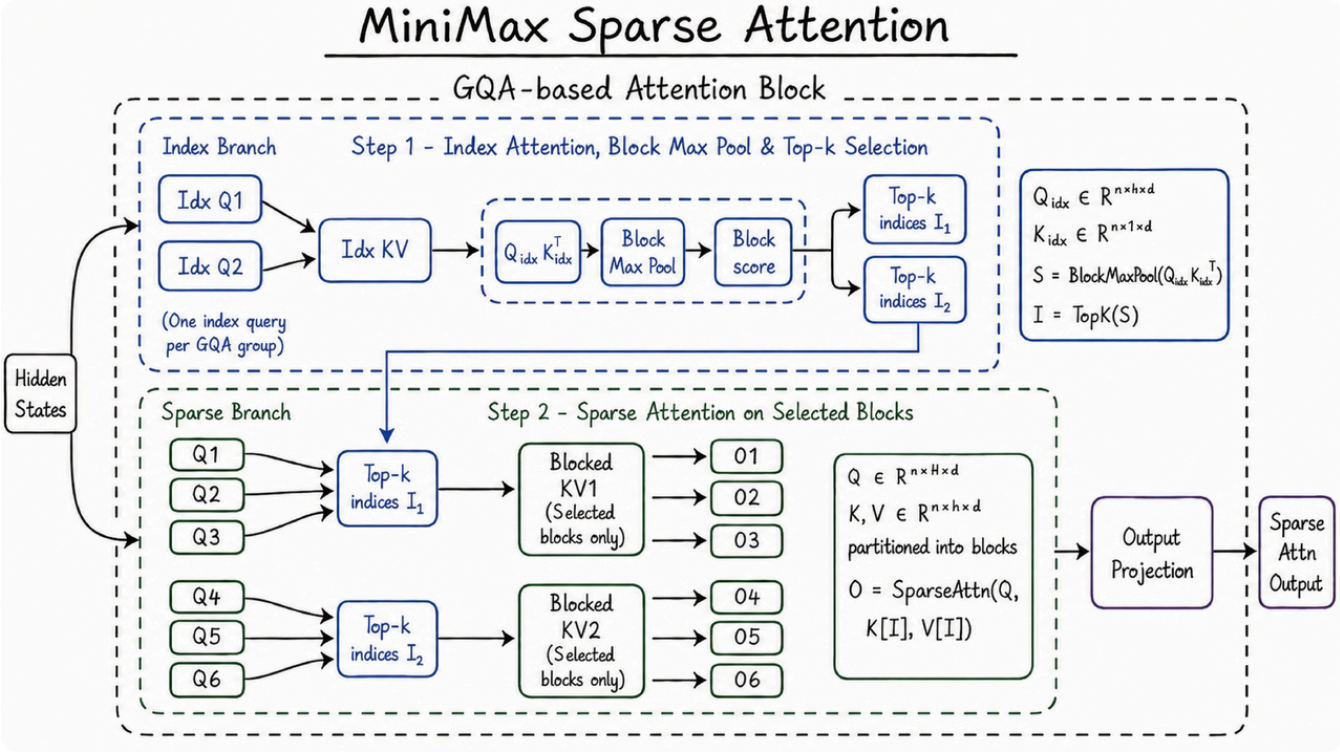
MSA: Architectural Innovation Enables Context Scaling

Solving more complex Agent tasks was one of the most important goals when training M3, and one of the biggest challenges involved was context scaling. To achieve real change, you have to start at the most fundamental level—the attention mechanism—and avoid the "inherent flaw" of full attention: quadratic computational complexity growth.

MSA is a clean and easily extensible new sparse attention architecture. It gives M3 a 1M context window and makes context truly another dimension that can be scaled.

Sparse attention mechanisms generally avoid the complexity-explosion problem by adding a pre-filtering stage. Compared with approaches like DSA and MoBA, MSA can partition the KV into blocks more precisely, achieving higher effective context coverage.

At the same time, we also optimized directly at the operator level, adopting a "KV outer gather Q" approach that uses KV blocks as the outer loop to aggregate the queries that hit them. Each block is read only once and memory access is contiguous; under M3's head configuration, the arithmetic intensity is significantly better than common methods—more than 4× faster than the open-source Flash-Sparse-Attention and flash-moba.



token compute is just 1/20 that of the previous-generation model. We achieved a speed up of more than 9× in the prefilling stage and more than 15x in the decoding stage. Moreover, across multiple ablations, MSA matched full attention on the vast majority of capabilities.

Frontier Coding and Agentic Capabilities

Coding and agentic capabilities are key areas of improvement for M3. Across internationally recognized benchmarks spanning software engineering and terminal execution, M3 reaches the frontier:

- SWE-Bench Pro: 59.0%
- Terminal-Bench 2.1: 66.0%
- SWE-fficiency: 34.8%
- KernelBench Hard: 28.8%
- MCP Atlas: 74.2%

Today, coding prowess increasingly depends on whether models can be trained using real-world user logic. Existing coding benchmarks often fail to fully capture real user experience.

Most current training and evaluation of code agents are based on the assumption of single-turn tasks. But real-world usage is not like this. Users often collaborate continuously within the same session: clarifying requirements, adjusting solutions, assigning tasks across contexts, and iterating over multiple rounds based on intermediate results.

To narrow the gap between benchmarks and real-world user experience, we built an interactive user simulator framework.

By simulating the behavioral patterns of real developers during collaboration, the framework exposes models during both training and evaluation to interaction scenarios that are much closer to production environments. It can simulate behaviors such as requirement elaboration, solution discussion, feedback-based correction, continuous task switching, and complex project iteration. As a result, the agent no longer merely executes instructions passively, but can actively collaborate with users to complete tasks.

The next generation of agentic coding will not be measured only by code generation, but also by long-term collaboration capability, planning ability, and the efficiency of human-agent collaboration. M3 scales up the data that truly matters for coding and agents, with

Multimodality: Interleaved Training, Continued Scaling

M3 is a model that has undergone mixed-modality training from Step 0. This native multimodal approach allows the semantic spaces of different modalities to merge more naturally and deeply.

Meanwhile, our extensive experiments show that interleaved data scales more easily than synthetic data. As a result, during the M3 cycle, we re-architected the entire text pretraining data pipeline, producing a large volume of interleaved data and incorporating it into model training.

Real-World Tasks

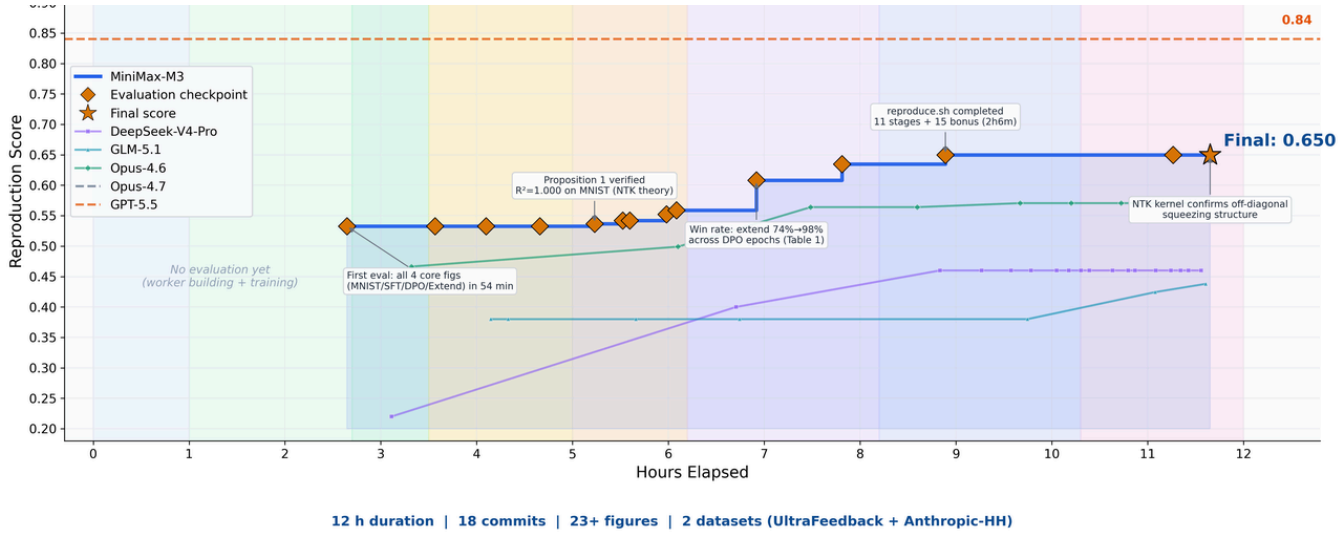
In our internal use and testing of M3, several real-world tasks left a strong impression.

Independent Paper Reproduction

As three essential capabilities for a frontier model, we wanted to see how 1M ultra-long context, top-tier coding and agent capabilities, and native multimodal capability would perform when brought together in a long thread to solve a complex task.

We gave M3 an ICLR 2025 Outstanding Paper Award-winning paper, *Learning Dynamics of LLM Finetuning*, and asked it to reproduce the paper independently. The paper studies the "learning dynamics" of large language models during fine-tuning. In the end, M3 ran autonomously for nearly 12 hours, independently producing 18 commits and 23 experimental figures throughout the process, and successfully completed the core experiments.

It not only successfully matched the trend of prediction-probability changes during the SFT stage, but also clearly observed the squeezing effect highlighted in the DPO experiments, and successfully verified the Extend mitigation method proposed in the original paper.



12 h duration | 18 commits | 23+ figures | 2 datasets (UltraFeedback + Anthropic-HH)

Multimodal capabilities were required to understand the curves, data, and formulas in the paper, while long context ensured that the paper, code, and experiment logs could all fit into the context window at once. Only with sufficiently strong coding and agent capabilities could the model complete the reproduction over a long thread, even with concurrent execution.

M3 was able to do it all.

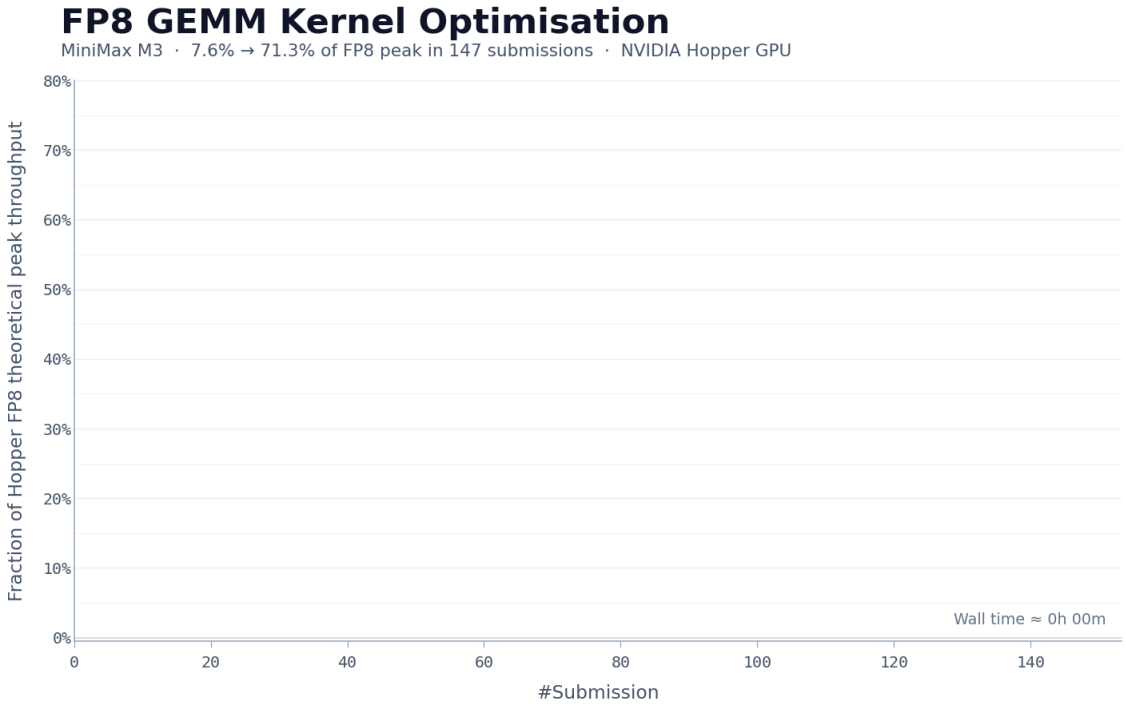
CUDA Kernel Optimization

FP8 matrix multiplication (GEMM) is one of the most compute-intensive parts of large model inference, and also one of the most difficult to optimize. Engineers must simultaneously handle multiple tightly coupled issues, including data layout, compute pipeline scheduling, and adaptation to hardware characteristics. On NVIDIA Hopper architecture GPUs, hand-writing a production-grade FP8 GEMM kernel typically requires one to two weeks of focused effort from an experienced team.

We used this task to evaluate M3's long-horizon autonomous iteration capability. We asked MiniMax M3 to optimize this kernel on NVIDIA Hopper architecture GPUs. The model started with only a task description, a benchmark evaluation script, and a Triton skeleton that could not run directly, with no reference high-performance implementation available. This meant the model could not take shortcuts by imitating an existing solution; it had to start from first principles and autonomously explore the optimization path.

Over the following approximately 24 hours of continuous execution, M3 completed 147 benchmark submissions and 1,959 tool calls. It independently went through the entire process from baseline implementation to production-grade optimization, including baseline implementation, autotune configuration generation, performance bottleneck diagnosis, CUDA Graph integration, persistent kernel rewriting, and host-side scheduling

In the end, after six landmark rounds of optimization, M3 improved Hopper FP8 hardware peak utilization from 7.6% in the first version to 71.3%, achieving a 9.4× speedup compared with the original version.



Beyond the metrics, the model's execution process is also worth noting. Except for Opus 4.7 and M3, most other models stopped making new progress within the first 30 submissions and exited on their own. M3's best solution, however, appeared on its 145th submission. Before that point, the model went through multiple performance plateaus where no further improvement was observed, yet it continued exploring different optimization directions.

The capabilities required here go beyond traditional code generation. The context produced by repeated tool calls is highly structured and dense, and this is where MSA's long-context attention allocation mechanism played an important role.

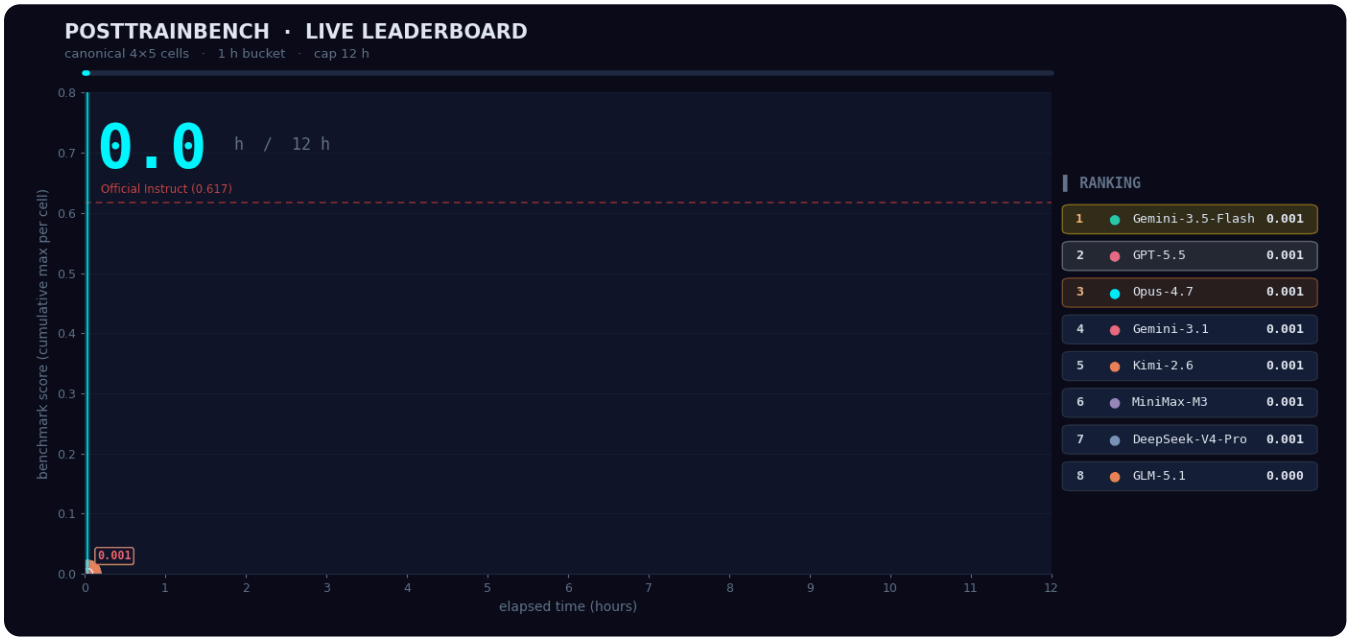
Letting M3 Train Models

In the CUDA operator optimization task, M3 demonstrated its long-horizon iteration capability on a single engineering task with a clear optimization objective and well-defined feedback signals. But real research work often does not have such a clear feedback structure; researchers are usually faced with more open-ended problems.

We wanted to understand how M3 performs in scenarios that require autonomous decision-making, so we tested it on PostTrainBench. The task was as follows: give M3 four

synthesis, training, evaluation, and iteration within 12 hours. The final goal was to enable these models to acquire basic capabilities across mathematical reasoning (AIME2025), tool calling (BFCL), scientific knowledge reasoning (GPQA Main), basic arithmetic reasoning (GSM8K), and code generation (HumanEval).

The entire “data synthesis → training → evaluation → iteration” process took place without any human intervention. The agent had to decide on its own what kind of data to synthesize, which training strategy to choose, and how to adjust the next round of plans based on evaluation results. M3 ultimately scored 0.37, slightly below Opus 4.7 (0.42) and GPT-5.5 (0.39), but clearly ahead of the other models.



MiniMax Code

With the release of M3, MiniMax Code has also been updated. As an agent product designed specifically for M3 and trained together with M3, MiniMax Code can fully leverage M3’s capabilities in long context, coding/agentic tasks, and native multimodality, making it the preferred agent to pair with MiniMax-M3.

For long-horizon complex tasks, MiniMax Code’s Agent Team can break large tasks down into multi-stage, concurrent, and dynamically adjustable workflows, which are then advanced collaboratively by a cluster of agents. Through a Producer + Verifier adversarial harness loop, the Agent Team can continuously produce, reflect, and correct itself during execution. It can run autonomously for days without human intervention and ultimately deliver high-quality results.

We have seen that Claude Code has also recently released Dynamic Workflows in a similar direction. Compared with Claude Code’s stronger emphasis on fixed orchestration based

while users can step in at any time to add requirements or correct the direction.

Thanks to M3's native multimodal capabilities, MiniMax Code also supports computer use. For example, a user can say on their phone: "Help me open the local ERP client and batch-enter invoice information based on this Excel spreadsheet." MiniMax Code will then automatically complete the required operations on the computer across applications, files, and systems.

MiniMax Code is built on a harness based on the outstanding open-source community projects OpenCode and Pi. We also plan to open-source this project in the future as a way to give back to the open-source community.

MiniMax Code desktop app: agent.minimaxi.com/download

MiniMax Code can be used with MiniMax Token Plans.

MiniMax Token Plan: Bringing Frontier Models to Developers' Daily Work

MiniMax M3 is a frontier model built to serve more users.

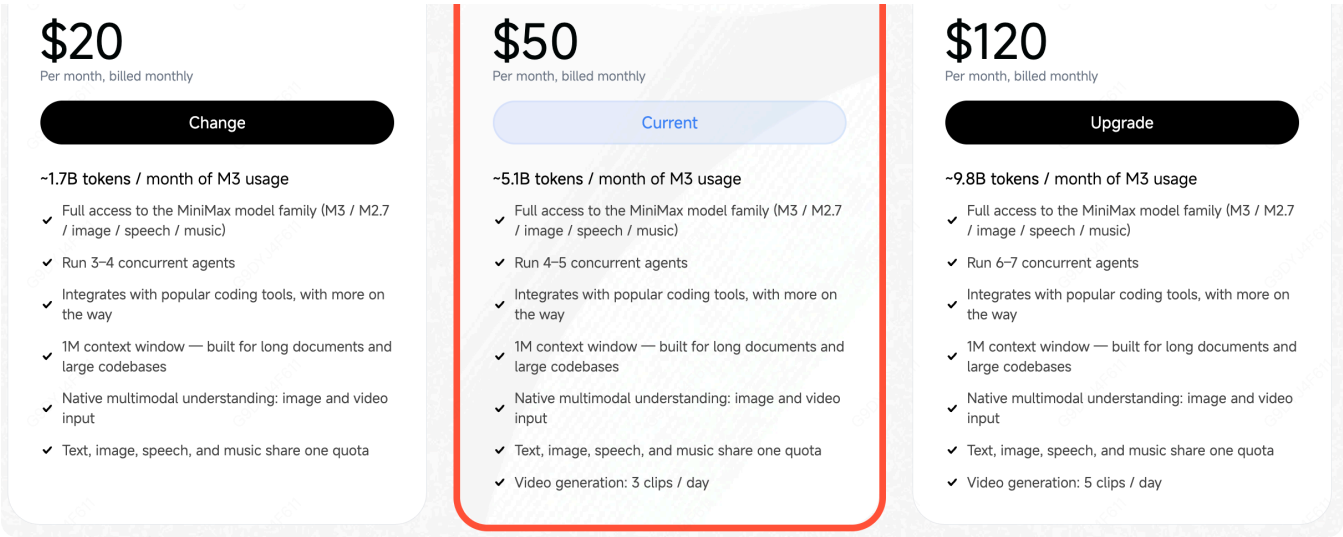
With this release, the MiniMax Token Plan has also been updated across three tiers:

- Plus $20/month: ~1.7B tokens / month of M3 usage
- Max $50/month: ~5.1B tokens / month of M3 usage
- Ultra $120/month: ~9.8B tokens / month of M3 usage

Among subscription plans at comparable price points, the MiniMax Token Plan offers one of the highest token quotas globally. Text, image, speech, and music all share the same usage pool.

All three tiers are fully available. Subscribe and start immediately!

Subscription link: platform.minimax.io/subscribe/token-plan



API

The M3 API is now available.

Pricing depends on input length: calls with **≤512K** input tokens are billed at the standard rate, covering the vast majority of conversation and coding scenarios, while calls **above 512K** are billed at a higher long-context rate, mainly intended for high-load scenarios such as ultra-long document parsing and full-repository code understanding.

M3 supports toggling thinking on or off. With thinking enabled, the model is suited to complex reasoning, agentic tasks, and long-horizon collaboration; with it disabled, it responds faster, suiting latency-sensitive scenarios such as conversation and code completion. The two modes share the same pricing and can be switched as needed at request time.

All prices can also be combined with two service levels: the default `standard` tier is suitable for regular requests; the `priority` tier (service_tier=priority) receives scheduling priority and more stable response latency under high-concurrency scenarios, making it suitable for SLA-sensitive industrial use cases. The priority channel is currently enabled through sales support and is expected to open to all users in a few days.

API guide: platform.minimax.io/docs/api-reference/api-overview

API Model Name	Input $/M tokens	Output $/M tokens	Caching Read $/M tokens
MiniMax M3 ≤512K	$0.60	$2.40	$0.12
MiniMax M3 512K-1M	$1.20	$4.80	$0.24

• MiniMax M3 (≤512k) — 50% off for 7 days

We will continue improving model serving stability and optimizing throughput. Over the next 10 days, we will release the model's technical report and open-source the corresponding model weights.

Today, models are being updated at such a fast pace that it is easy to forget this is still a matter of steady, incremental progress. It follows its own objective laws, and it rewards teams that move forward solidly in accordance with those laws. Just as we believed at the very beginning of our founding, we will do our utmost to continuously improve the intelligence of our models and make them available to more users.

Thank you for your trust, suggestions, and criticism.

Intelligence with Everyone!

Coding									
SWE-Bench Verified	80.5	79.9	87.6	82.9	80.6	79.6	80.6	-	80.2
SWE-Bench Pro	59.0	56.2	64.3	58.6	54.2	-	55.4	58.4	58.6
Terminal Bench 2.1	66.0	51.1	66.1	78.2	70.3	-	-	-	-
SWE Atlas-QnA	37.9	11.29	45.16	45.43	13.5	31.20	-	-	-
nl2repo	42.13	34.99	56.28	52.9	21.62	-	35.5	41	42.8
SWE Atlas-Test Writing	30.83	18.89	38.21	42.59	29.84	31.76	-	-	-
SWE-fficiency	34.8	13.98	42.2	46.6	19.7	-	-	-	-
LiveSQLBench	40.17	33.17	41.00	40.17	39.83	-	-	-	-
CL-bench	20.48	15.38	22.92	25.38	21.06	-	-	-	-
VIBE-V2	50.12	37.89	55.87	50.50	28.00	-	-	-	-
SVG-Bench	63.7	48.0	62.3	58.2	59.2	-	-	-	-
PostTrainBench	37.1	13.1	42.4	39.3	15.2	-	-	-	-
KernelBench Hard	28.8	10.5	30.7	20.9	18.6	-	-	-	-
PaperBench	52.6	30.6	58.5	57.5	46.7	-	-	-	-
Cowork (Agent)									
BrowseComp	83.52	76.3	79.3	84.4	85.9	74.7	83.4	79.3	83.2
DRACO	73.23	66.77	77.7	-	-	75.8	-	-	-
GDPval rubrics	74.78	66.44	79.8	80.66	57.82	75.65	70.32	68.26	65.12
BankerToolBench	76.12	63.89	81.34	70.04	67.03	-	-	-	-
OfficeQA Pro	45.1	-	43.6	52.6	18.1	-	-	-	-
SpreadSheetBench-v1	89.35	84.92	88.49	88.11	56.06	-	84.9	85.2	84.5
YC-Bench	2.10M	0	2.19M	1.28M	1.05M	-	-	-	-
LOCA-Bench (256k)	49.3	0	57	-	-	-	-	-	-
MCP Atlas	74.2	49.4	77	75.3	69.2	61.3	73.6	71.8	66.6
Apex-Agents	27.7	5.6	37.2	41.7	33.4	26.2	-	-	-
Claw-Eval	74.5	49.7	71.6	-	57.8	68.3	58.4	62.7	61.5
GUI									
OSWorld-Verified	70.06	-	82.8	78.7	76.2	72.5	80.6	-	73.1
MultiModal									
OmniDocBench	91.6	-	89.3	87.5	88.1	86.9	-	-	-
MMMU-Pro	78.1	-	77	81.2	80.5	74.5	-	-	79.4
Video-MMMU	84.6	-	83	86.4	87.9	-	-	-	-
VideoMME (w/ sub)	85.4	-	-	89.4	87.9	-	-	-	-
Reasoning									
IMO 2025	35 / 42	-	-	-	-	-	-	-	-
USAMO 2026	36 / 42	-	52.8%	98.21%	74.40%	-	-	-	-

Evaluation Methodology

test was run 4 times and the average was taken.

SWE-Bench Pro: Tested on internal infrastructure using Claude Code as the scaffolding. Testing logic is aligned with the official evaluation.

Terminal-bench 2.1: Evaluated on internal infrastructure with a sandbox configured as 8C16G, a timeout of 2 hours, and max output tokens set to 128K, using Terminus 2 as the scaffolding. Scores for GPT-5.5, Gemini 3.1 Pro, and Claude Opus 4.7 are taken from the official Terminal-bench 2.1 leaderboard; all other models were tested via official API on the same infrastructure.

SWE Atlas-Codebase QNA: Evaluated on internal infrastructure with a sandbox configured as 4C8G and a 3-hour timeout. Scores for Claude Sonnet 4.6, GPT-5.5, and Gemini 3.1 Pro are taken from labs.scale.com. Claude Opus 4.7, MiniMax-M2.7, and MiniMax-M3 used Mini-SWE-Agent as scaffolding, with evaluation logic aligned with the official method.

NL2Repo: Scores for DeepSeek-V4-pro, Kimi-k2.6, and GLM-5.1 are taken from https://qwen.ai/blog?id=qwen3.7
. Other models were evaluated on internal infrastructure with a sandbox configured as 1C2G and a 4-hour timeout. Claude Opus 4.7, MiniMax-M2.7, MiniMax-M3, and Gemini 3.1 Pro used Claude Code scaffolding; GPT-5.5 used Codex scaffolding. To prevent potential model "hacks," the following modifications were made based on official evaluation logic: (1) prompts included constraints prohibiting the model from using external information via `git clone`, `pip install`, etc.; (2) at the scaffolding level, system-monitored Bash commands executed by the model were analyzed for potential cheating. Commands determined as cheating were intercepted, and the model was warned to complete the task within the restricted environment.

SWE Atlas-Test Writing: Scores for GPT-5.5, Claude Sonnet 4.6, and Gemini 3.1 Pro are from labs.scale.com. Claude Opus 4.7, MiniMax-M2.7, and MiniMax-M3 were evaluated on internal infrastructure using Claude Code scaffolding with a sandbox of 4C8G and a 3-hour timeout, aligned with official logic, run 4 times and averaged.

SWE-fficiency: Evaluated on internal infrastructure using the open-source SWE-fficiency dataset and workflow. Sandbox: 1C2G, timeout: 2 hours. Claude Code used as scaffolding; scores are from internal testing.

LiveSQLBench: Evaluated on internal infrastructure using the open-source LiveSQLBench-Base-Full v1 dataset (600 questions / 22 PostgreSQL databases) and official workflow. Claude Code used as scaffolding, with task description prompts

VIBE-V2: Internal benchmark covering pure front-end and full-stack Web/Android/iOS projects, task type: build from scratch. Claude Code used as scaffolding, with Agent-as-a-Verifier paradigm for automated verification of program interaction logic and visual output. Scores computed via unified pipeline including requirement set, containerized deployment, and dynamic interaction environment, averaged over 3 runs.

SVG-Bench: Internal benchmark, input types: text and images, tasks: build from scratch or edit based on existing assets. Claude Code used as scaffolding, VLM used to verify rendering accuracy, averaged over 3 runs.

CL-bench: Evaluated on internal infrastructure using open-source CL-bench data and rubrics. Evaluation setup fully aligned with official procedure. Scores from internal testing.

PostTrainBench: Evaluated on Claude Code using Ralph-Loop mechanism for 12 hours, testing 4 Base Models across 5 benchmarks not requiring LLM-As-Judge (AIME2025, BFCL, GPQA Main, GSM8K, HumanEval).

Kernelbench-Hard: Evaluated on Claude Code on NVIDIA Blackwell architecture GPUs with CUDA capability sm_120. Score per question = Agent's submitted operator TFLOPs relative to theoretical peak of current hardware; benchmark score = average of 9 questions.

PaperBench: Evaluated on Claude Code using Ralph-Loop mechanism for 12 hours. Dataset: 19 papers reproducible without external API. Rubrics: official open-source human expert rubrics. Scoring model: Opus-4.6.

GPDval-Rubrics: Internal evaluation using cases from the public GDPval dataset, pointwise scoring based on public rubrics, environment aligned with GDPval-AA scaffolding.

BrowseComp: Uses same agent framework as WebExplorer (Liu et al., 2025). When token usage exceeds 64K, all history is discarded.

DRACO: MiniMax M3 results evaluated using internal scaffolding (accessible via Deep Research Skill in MiniMax Code). Scoring based on official rubrics per question, final score = average across all questions. Scoring model: Claude Opus 4.6. Claude Opus 4.7 results taken from Opus 4.7 model card.

BankerToolBench: Tested on public BankerToolBench dataset. All models except GPT-5.5 used Claude Code scaffolding; GPT-5.5 used Codex. Scoring based on dataset rubrics, using MiniMax M2.7 as scoring model.

answers.

SpreadSheetBench-v1: Evaluated on public dataset using Claude Code scaffolding.

YC-Bench: Evaluated using official YC-Bench codebase and configuration, environment aligned with official setup. Metric: final assets (fund).

LOCA-Bench (256k): Evaluated using official LOCA-bench codebase, official react mode, Environment Description Length = 256k.

MCP Atlas: Evaluated using official MCP Atlas codebase. Public Set scores using Gemini 2.5 Pro as scoring model, aligned with official model.

Apex-Agents: Uses archipelago codebase, ReAct Toolbelt framework, scoring model Claude Sonnet 4.6.

Claw-Eval: Evaluated using official Claw-Eval codebase, General Task Group (161 tasks), scoring model Gemini 3.0 Flash, aligned with official model. Metric: Pass³ score.

OSWorld-Verified: Tested on 361 samples from nogdrive collection using OSWorld-Verified official codebase (testing script soon to be open-sourced). M3 uses relative coordinates 0–1000, image resolution 1920×1080, Max Steps = 200. Increasing Max Steps from 100 → 200 improved task completion rate from 68.70% → 70.06%.

OmniDocBench: Image long edge max = 3584 pixels, using public OmniDocBench v1.5 dataset and official evaluation logic. Added reasonable formatting constraints on top of official prompts. Gemini 3.1 Pro, GPT-5.5, Claude Opus 4.7 used default API parameters.

MMMU Pro: Aligned with official evaluation. Prompt enforces format constraint on model's last line for easier parsing.

VideoMMMU: Video frame rate = 1 FPS, max 512 frames, single-frame long edge 672–1008 pixels. Official VideoMMMU prompt used, LLM-as-a-Judge scoring. MiniMax M3: max output tokens = 32K, temperature = 1.0, top_p = 0.95. External models: max output tokens = 64K, temperature = 0.7, top_p = 0.95, highest thinking mode.

Video-MME: Video frame rate = 1 FPS, max 1024 frames (*external API limit 640 frames; MiniMax M3 scored 84.6 at 512 frames), single-frame long edge 336–672 pixels, subtitles inserted every 30 seconds interleaved into frames. Official Video-MME prompt used, LLM-as-a-Judge scoring. MiniMax M3: max output tokens = 16K, temperature = 1.0, top_p = 0.95. External models: max output tokens = 64K, temperature = 0.7, top_p = 0.95, highest thinking mode. *Note: Claude Opus 4.7 API error rate >20%, results not reported.

models graded using human expert rubrics (GPT-5.4 high reasoning effort, Gemini 3.1 Pro high reasoning effort) → (3) dual judges take minimum as final score. M3 evaluation: 512k max output tokens, temperature = 1.0, test-time-scaling framework up to 10 iterations. Other closed-source model metrics: avg@k results.